

OPERATOR PRECEDENCE & ASSOCIATIVITY

Operator Precedence (What runs first)

Higher precedence = executed earlier:

- 1) $() \rightarrow$ Parenthesis
- 2) $** \rightarrow$ Exponentiation / Power
- 3) $*, /, \% \rightarrow$ Multiply, Divide, Modulus
- 4) $+, - \rightarrow$ Add, Subtract

Associativity (What runs first when same precedence)

Left to Right: $+, -, *, /, \%$

Right to Left: $**, =$

PRACTICE QUESTIONS

Q1: $5 + 3 * 2$

Q2: $(5 + 3) * 2$

Q3: $10 - 4 + 2$

Q4: $10 - (4 + 2)$

Q5: $8 / 2 * 4$

Q6: $8 / (2 * 4)$

Q7: $2 ** 3 ** 2$

Q8: $(2 ** 3) ** 2$

Q9: $10 \% 4 + 2$

Q10: $10 + 4 \% 2$

Q11: $100 / 5 ** 2$

Q12: $4 + 3 * 2 - 1$

Q13: $6 + 2 * (3 + 1)$

Q14: $(6 + 2) * (3 + 1)$

Q15: $18 / 3 / 3$

Q16: $18 / (3 / 3)$

Q17: $10 - 2 ** 2$

Q18: $(10 - 2) ** 2$

Q19: $5 * 2 \% 3$

Q20: $5 \% 2 * 3$

Q21: $2 + 3 * (4 ** 2 - 6) / 2$

Q22: $((10 + 2) * 3) ** 2 \% 5$

Q23: $20 - 4 * 2 ** 2 + 1$

Q24: $100 / (5 + 5) * 2 ** (2 + 1)$

Q25: `(8 + 2) ** 2 / 2 - 5 % 3`

Q26: `a = b = c = 5;`

Q27: `let a, b, c;`

`a = b = c = 10;`

`console.log(a, b, c);`

a) 10 10 10

b) undefined undefined 10

c) Syntax Error

d) NaN NaN NaN

Q28: `let x = 5;`

`let y = 10;`

`let z = (x = y);`

`console.log(z);`

a) 5

b) 10

c) undefined

d) Error

Q29: `let a = 5;`

`let b = a = 7;`

`console.log(a, b);`

a) 5 5

b) 7 5

c) 7 7

d) undefined undefined

Q30: what will be the output `"5" + "5"`

Q31: what will be the output `"5" + 5`

Q32: what will be the output `5 + "5"`

Q33: what will be the output `"5" - "5"`

Q34: what will be the output `"5" * "2"`

Q35: what will be the output `"5" / "2"`

Q36: what will be the output `"5" ** "2"`

Q37: what will be the output `"5" % "2"`

PRIMITIVE DATA TYPES

Number

- Represents both integers and floating-point numbers.
- Includes special values like NaN, Infinity, and -Infinity.
- `typeof 42` → "number"
- NaN stands for "Not a Number" but is still of type "number"

- Division by zero gives Infinity, not an error

String

- Represents textual data, enclosed in single, double, or backticks (`).
- Strings are immutable.
- `typeof "hello" → "string"`
- String indexing like `str[0]` is allowed (but strings are immutable)
- Template literals allow embedded expressions: ``Hello ${user}``

Boolean

- Represents a logical value: true or false
- Often used in conditions and comparisons
- `typeof true → "boolean"`
- Common use in if, while, and ternary expressions

Undefined

- A variable that is declared but not assigned a value is undefined.
- JavaScript automatically assigns undefined to uninitialized variables.
- `typeof undefined → "undefined"`
- Returned by functions with no return statement

Null

- Represents the intentional absence of any object value.
- Manually assigned by the developer.
- `typeof null → "object"` (known JavaScript bug)
- Use null to reset or clear a variable.

Note:

- All primitive data types are immutable.
- They are copied by value, not by reference.

Convert String → Number

1. `parseInt(str)`
2. `parseFloat(str)`
3. Unary plus `+str`

Convert Number → String

1. `num.toString()`
2. `num + ""`

PRACTICE QUESTIONS

Q1: What are primitive data types in JavaScript? List them.

Q2: What is the difference between null and undefined?

Q3: Is NaN a primitive data type in JavaScript? Explain.

Q4: What data type is returned by typeof null? Why is that considered weird?

Q5: What is the output of typeof "42" and typeof 42? Explain the difference.

Q6: Convert a string "123" into a number and print the result and its type.

Q7: What will be the output of this code?

```
let x;
```

```
console.log (typeof x);
```

Q8: What is the output and type of this?

```
let a = null;
```

```
console.log(typeof a);
```

Q9: What's the result of adding a string and a number in JS?

```
let result = "Age: " + 25;
```

```
console.log(result);
```

Q10: What will this log, and why?

```
let str = "Hello";
```

```
str[0] = "h";
```

```
console.log(str);
```

Q11: Which primitive type would you use to represent:

- A person's name
- Whether someone is logged in
- A large bank balance
- A missing value

Q12: Which of the following is a primitive data type in JavaScript?

- a) Object
- b) Array
- c) Number
- d) Function

Q13: What will typeof null return in JavaScript?

- a) "null"
- b) "undefined"
- c) "boolean"
- d) "object"

Q14: What is the result of typeof NaN?

- a) "NaN"
- b) "undefined"
- c) "number"
- d) "object"

Q15: Which one of the following is not immutable?

- a) String
- b) Number
- c) Object

Q16: Is NaN a primitive?

- a) Yes
- b) No

Q17: You can use typeof to check for primitive types ?

Q18: Why does typeof null return "object" ?

VARIABLES

- Variables are named containers used to store data in JavaScript.
- let and var can be reassigned, but const cannot.
- Only var allows redeclaration in the same scope.
- var declarations are hoisted to the top of their scope and initialized with undefined.
- let and const are hoisted but accessing them before declaration causes a ReferenceError.
- Global Scope: Declared outside functions or blocks.
- Function Scope: Variables declared with var inside a function.
- Block Scope: Variables declared with let or const inside {} blocks.
- The time between entering the scope and the actual declaration of a let or const variable is called the Temporal Dead Zone. Accessing variables in the TDZ throws a ReferenceError.
- const variables cannot be reassigned, but if they reference objects or arrays, their contents can be changed.

Naming Rules and Conventions

- Cannot start with a number
- Must begin with a letter, underscore (_), or dollar sign (\$)
- JavaScript are case-sensitive
- Use camelCase by convention
- Prefer const by default – use let if you need reassignment.
- Avoid var in modern JavaScript.
- Always declare variables at the top of their scope.
- Use meaningful, descriptive variable names.

PRACTICE QUESTIONS

Q1: What are the different ways to declare variables in JavaScript?

Q2: Declare a variable called username.

Q3: Declare a variable myName & assign it a string that represents your Name.

Q4: Write script to

- a) Declare a JS variable, called message.
- b) Assign "Hello World" to variable message

c) Display the message in alert box.

Q5: Write a script to save student's bio data in JS variables and show the data in alert boxes.

Q6: Declare a variable called age & assign to it your age. Show your age in an alert box.

Q7: Declare a variable called birthYear & assign to it your birth year.

Q8: Write a script to store in variables the following information:

a) Visitor's name

b) Product title

c) Quantity i.e. how many products a visitor wants to order

Q9: Declare 3 variables in one statement.

Q10: Declare 5 legal & 5 illegal variable names.

Q11: Write a program that take two numbers & add them in a new variable.

Q12: Repeat task1 for subtraction, multiplication, division & modulus.

Q13: Cost of one movie ticket is 600 PKR. Write a script to store ticket price in a variable & calculate the cost of buying 5 tickets to a movie.

Q14: Write a script to display multiplication table of any number.

Q15: Assume we have 10 US dollars & 25 Saudi Riyals. Write a script to convert the total currency to Pakistani Rupees. Perform all calculations in a single expression.

Q16: The Age Calculator: Forgot how old someone is? Calculate it!

a) Store the current year in a variable.

b) Store their birth year in a variable.

c) Calculate his / her age based on the stored values.

Q17: The Lifetime Supply Calculator:

a) Store your favorite snack into a variable

b) Store your current age into a variable.

c) Store a maximum age into a variable.

d) Store an estimated amount per day (as a number).

e) Calculate how many would you eat total for the rest of your life.

Q18: What is the difference between var, let, and const?

Q19: Can you reassign a value to a const variable?

Q20: What is the scope of a var variable?

Q21: What happens if you declare a variable without var, let, or const?

Q22: What is the default value of an uninitialized variable?

Q23: How do you check the type of a variable in JavaScript?

Q24: What is block scope, and how do let and const handle it?

Q25: Explain temporal dead zone (TDZ) in the context of let and const.

Q26: Does JavaScript support typed variables?

Q27: How are variables stored in memory (stack vs. heap)?

Q28: What is the output and why?

```
console.log(x);
```

```
var x = 5;
```

Q29: What is the output and why?

```
console.log(y);
```

```
let y = 5;
```

Q30: What happens if you declare the same variable name twice using let in the same block?

Q31: Can you declare a const variable without assigning a value?

Q32: What will this output? Why?

```
var a = 10;
```

```
{  
    var a = 20;  
    console.log(a);  
}  
console.log(a);
```

DIALOG BOXES

Alert:

- It is used to display an informational message to the user.
- Shows a modal dialog with a message and an OK button.
- Does not return any value.

Prompt:

- It is used to get input from the user via a text box.
- 1st: Message to display.
- 2nd (optional): Default text inside the input field.
- The text entered by the user (as a string).
- null if the user clicks Cancel.

Confirm:

- It is used to ask the user to confirm an action (Yes/No or OK/Cancel).
- true if the user clicks OK.
- false if the user clicks Cancel.

Note:

- These are blocking functions - they pause JavaScript execution.
- Their design and behavior cannot be styled or changed with CSS.

- Not ideal for modern UI/UX - use HTML/CSS modals in production apps.

PRACTICE QUESTIONS

- Q1: Write a script to greet your visitor using alert box (Hint: Use line break).
- Q2: What is the purpose of the alert() function in JavaScript?
- Q3: What kind of value does the prompt() function return?
- Q4: How does a confirm() dialog box differ from an alert() box?
- Q5: What happens when the user clicks "OK" in a prompt() dialog?
- Q6: Can you use alert() to stop the execution of a script temporarily?
- Q7: What are the default buttons shown in a confirm() dialog?
- Q8: What is returned by a confirm() dialog when the user presses "Cancel"?
- Q9: Can a prompt() dialog have a default value?
- Q10: What data type is returned by a confirm() dialog?
- Q11: How many values does prompt() accept?
- Q12: What happens if a user presses "Cancel" on a prompt() dialog?
- Q13: Can alert() return any value?
- Q14: What will prompt("Enter your name:") return if the user clicks "OK" without typing anything?
- Q15: What is the output of alert(prompt("Enter a value:")) if the user enters "123"?
- Q16: Does confirm() stop the script execution until the user responds?
- Q17: What will typeof prompt("Enter something:") return?
- Q18: Can you show multiple alert() boxes one after another?
- Q19: Use prompt() to collect two numbers from a user and alert() the sum.

STRING METHODS

- a) charAt(index) → Returns the character at the specified index
- b) charCodeAt(index) → Returns the Unicode of the character at the given index
- c) concat(str1, str2, ...) → Joins two or more strings.
- d) includes(substring) → Checks if a string contains the specified substring. Returns true or false.
- e) indexOf(substring) → Returns the first index of the substring. Returns -1 if not found.
- f) lastIndexOf(substring) → Returns the last index of the substring.
- g) slice(start, end) → Extracts a section of a string and returns it as a new string.
- h) substring(start, end) → Similar to slice, but doesn't accept negative indexes.
- i) toLowerCase() → Converts the entire string to lowercase.
- j) toUpperCase() → Converts the entire string to uppercase.

- k) `trim()` → Removes whitespace from both ends of a string.
- l) `trimStart()` / `trimLeft()` → Removes whitespace from the start of a string
- m) `trimEnd()` / `trimRight()` → Removes whitespace from the end of a string.
- n) `replace(search, replacement)` → Replaces the first match with a new substring.
- o) `replaceAll(search, replacement)` → Replaces all matches of a string.
- p) `split(separator)` → Splits a string into an array using a separator.
- q) `repeat(count)` → Returns a new string with the original repeated count times.
- r) `padStart(targetLength, padString)` → Pads the start of the string with another string until it reaches the given length.
- s) `padEnd(targetLength, padString)` → Pads the end of the string with another string until it reaches the given length.
- t) `Length` → it gives a total length of a string.

PRACTICE QUESTIONS

- Q1: What is the output of `"JavaScript".charAt(4)`?
- Q2: What index should be used with `charAt()` to get the first character?
- Q3: `"Hello".charAt(10)` returns what?
- Q4: What will `"12345".charAt(2)` return?
- Q5: Which method returns the character at a given index?
- Q6: Write a code to get the last character of a string using `charAt()`.
- Q7: Fill in the blank: `"Test".charAt(____)` returns "s".
- Q8: How does `charAt()` differ from using square brackets (e.g., `str[2]`)?
- Q9: What is the output of `"A".charCodeAt(0)`?
- Q10: `"abc".charCodeAt(1)` returns what number?
- Q11: Which method gives the Unicode value of a character at a given index
- Q12: Write code to get the Unicode of the last character in "hello".
- Q13: True or False: `charCodeAt()` can return NaN.
- Q14: What does `"OpenAI".charCodeAt(100)` return?
- Q15: What's the difference between `charAt()` and `charCodeAt()`?
- Q16: What is the output of `"Hello".concat(" ", "World")`?
- Q17: Fill in the blank: `"JS".concat("Script") = _____`.
- Q18: Can `concat()` be used to join more than 2 strings?
- Q19: Is `concat()` destructive (does it modify the original string)?

Q20: Write a code to combine "I", "am", "GPT" into one string using concat().

Q21: True or False: "A".concat("B", "C") returns "ABC".

Q22: What does "Hello".includes("lo") return?

Q23: Can includes() be case-sensitive?

Q24: What will "apple".includes("pp") return?

Q25: What's the type of the value returned by includes()?

Q26: Can includes() accept a second argument to start searching from a specific index?

Q27: What does "abcdef".includes("c", 3) return?

Q28: What does "hello".indexOf("e") return?

Q29: What does "hello world".indexOf("world") return?

Q30: What is returned when the substring is not found?

Q31: Can indexOf() take a second parameter? What does it do?

Q32: What does "banana".indexOf("a", 2) return?

Q33: Write code to check if "test" contains "e" using indexOf.

Q34: What's the difference between includes() and indexOf()?

Q35: Is the result of indexOf() always a number?

Q36: What will "JavaScript".slice(0, 4) return?

Q37: What does "HelloWorld".slice(5) return?

Q38: Can slice() accept negative values?

Q39: What will "abcdef".slice(-3) return?

Q40: What does "abcdef".slice(1, 4) return?

Q41: What happens if start > end in slice()?

Q42: Write code to extract the last 4 characters of "developer".

Q43: What will "abcdef".substring(1, 4) return?

Q44: How does substring(2, 5) differ from slice(2, 5)?

Q45: Is toUpperCase() destructive?

Q46: What does "JavaScript123".toUpperCase() return?

Q47: Capitalize the first letter of the string.

Q48: What will `"HELLO".toLowerCase()` return?

Q49: Can you chain `toLowerCase()` with `includes()`?

Q50: What does `" hello ".trim()` return?

Q51: Is `trim()` destructive?

Q52: What is returned by `" ".trim()`

Q53: Does `trim()` remove spaces inside the string?

Q54: Can `trim()` be chained with `toUpperCase()`?

Q55: What's the result of:

```
let str = "  code ";
console.log(str.trim() + "wars");
```

Q56: What does `"apple".replace("p", "b")` return?

Q57: Does `replace()` affect all matches?

Q58: Can you use regex in `replace()`?

Q59: Write code to replace spaces with underscores in `"hello world"`.

Q60: Is `replace()` destructive?

Q61: What does `"a,b,c".split(",")` return?

Q62: What does `"one two three".split(" ")` return?

Q63: Is the return type of `split()` always an array?

Q64: What does `"5".padEnd(3, "0")` return?

Q65: How can you format `"7"` as `"700"` using `padEnd()`?

Q66: `"abc".padEnd(10, "123")` return?

Q67: True or False: `padEnd()` modifies the original string.

Q68: What does the `.repeat()` method do in JavaScript, and what type of value does it return?

Q69: What does the `.length` property return when used with a string?

Q70: What will be the output:

```
let str = "hello world";
str.length = 0;
console.log(str)
```

MATH OBJECT

- a) `Math.round(x)` → Rounds `x` to the nearest integer.
- b) `Math.floor(x)` → Rounds `x` down to the nearest integer.
- c) `Math.ceil(x)` → Rounds `x` up to the nearest integer.

- d) `Math.trunc(x)` → Truncates the decimal part, returns the integer part of `x`.
- e) `Math.pow(x, y)` → Returns `x` raised to the power of `y`.
- f) `Math.sqrt(x)` → Returns the square root of `x`.
- g) `Math.cbrt(x)` → Returns the cube root of `x`.
- h) `Math.min(...args)` → Returns the smallest number from arguments.
- i) `Math.max(...args)` → Returns the largest number from arguments.
- j) `Math.random()` → Returns a random number between 0 (inclusive) and 1 (exclusive).

PRACTICE QUESTIONS

- Q1: What is `Math.round(4.4)`?
- Q2: How is `Math.round(4.5)` different from `Math.floor(4.5)`?
- Q3: Will `Math.round(-2.8)` round up or down?
- Q4: Does `Math.round()` always return an integer?
- Q5: What is `Math.floor(7.9)`?
- Q6: What does `Math.floor(-2.1)` return?
- Q7: What's the difference between `Math.floor(3.1)` and `parseInt(3.1)`?
- Q8: What is the result of `Math.ceil(4.01)`?
- Q9: What will `Math.ceil(-4.1)` return?
- Q10: How is `Math.ceil()` different from `Math.round()`?
- Q11: What is `Math.trunc(3.8)`?
- Q12: What does `Math.min(3, 5, -2)` return?
- Q13: What is the result of `Math.min()` with no arguments?
- Q14: What is `Math.max(2, 100, 0)`?
- Q15: What does `Math.random()` return?
- Q16: Is the value from `Math.random()` ever exactly 1?
- Q17: How can you generate a random number between 1 and 100?
- Q18: How to get a random integer using `Math.random()` and `Math.floor()`?
- Q19: What is `Math.pow(2, 3)`?
- Q20: What is `Math.sqrt(9)`?
- Q21: What is `Math.cbrt(27)`?
- Q22: Create a game that takes a number from user and if user number matched then show the result.

TRUTHY AND FALSY VALUES IN JAVASCRIPT

- In JavaScript, every value is either truthy or falsy when evaluated in a boolean context.

Falsy Values:

- 1) false
- 2) 0
- 3) -0
- 4) ""
- 5) null
- 6) undefined
- 7) NaN

Truthy Values:

- Any value that is not falsy is considered truthy.

Boolean Method in JavaScript:

- In JavaScript, Boolean() is a built-in function used to convert any value to a boolean (true or false)
- It converts any value to:
 - true → if the value is truthy
 - false → if the value is falsy

COMPARISON OPERATORS

- 1) == Equal to (loose equality)
- 2) === Strict equal to (type + value)
- 3) != Not equal to (loose)
- 4) !== Strict not equal to
- 5) > Greater than
- 6) < Less than
- 7) >= Greater than or equal to
- 8) <= Less than or equal to

Note:

- Prefer === and !== for cleaner, more predictable code.
- Avoid relying on == unless you fully understand type coercion.
- Use comparisons inside conditions (if, loops, etc.)

PRACTICE QUESTIONS

Q1: What is the result of 5 == "5"?

- a) true
- b) false
- c) undefined
- d) error

Q2: Which operator checks for strict equality?

- a) ==
- b) =
- c) ===
- d) !==

Q3: What does `null == undefined` evaluate to?

- a) true
- b) false
- c) error
- d) NaN

Q4: What is the result of `"5" === 5`?

- a) true
- b) false
- c) error
- d) NaN

Q5: Which operator checks value and type?

- a) ==
- b) ===
- c) !=
- d) >=

Q6: `10 > 5` will return:

- a) true
- b) false
- c) 5
- d) 10

Q7: `5 !== "5"` evaluates to:

- a) true
- b) false
- c) undefined
- d) null

Q8: Which expression is false?

- a) `"a" < "b"`
- b) `5 >= 5`
- c) `0 == false`
- d) `null === undefined`